```
            dfs.setDecimalSeparator('.');
            dfs.setGroupingSeparator(',');
            DecimalFormat f = new DecimalFormat("######.###");
            f.setDecimalFormatSymbols(dfs);

            for (Shape shape : shapes) {
                System.out.println(shape + ", εμβαδόν: " +
f.format(shape.getArea()));
            }
        }
}
```

Κώδικας 13.20 *Η εφαρμογή CalcAreas.*

Σύμφωνα με την εκφώνηση, η εφαρμογή μας φορτώνει οκτώ γεωμετρικά σχήματα σε ένα ArrayList και στη συνέχεια εμφανίζει την String αναπαράσταση και το εμβαδόν καθενός. Παρατηρήστε πως κατά τη διαμόρφωση της εξόδου καθορίζουμε το διαχωριστικό δεκαδικών να είναι ο χαρακτήρας '.' και χιλιάδων να είναι ο χαρακτήρας ','.

13.4.2 Η κλάση Quiz

Να υλοποιήσετε την κλάση Quiz που διαθέτει τις ακόλουθες λειτουργίες:

1. Παρέχει λειτουργία πρόσθεσης ερωτήσεων.
2. Υποστηρίζει ερωτήσεις σωστού-λάθους και πολλαπλών επιλογών
3. Κατά την εκτέλεσή της, υποβάλλει τον χρήστη σε τεστ με βάση τις καταχωρημένες ερωτήσεις.
4. Για κάθε ερώτηση επιβεβαιώνει τη σωστή απάντηση ή αν ο χρήστης απαντήσει λανθασμένα, εμφανίζει τη σωστή απάντηση.

Λύση

Από την εκφώνηση είναι φανερό πως έχουμε να διαχειριστούμε ερωτήσεις δύο τύπων, σωστού-λάθους και πολλαπλών επιλογών. Και οι δύο τύποι όμως, παρότι εξειδικεύονται, παραμένουν ερωτήσεις. Επομένως, ένας σχεδιασμός με γονική κλάση, αυτή των ερωτήσεων και παράγωγες τις κλάσεις των ερωτήσεων σωστού-λάθους και πολλαπλών επιλογών θα φανεί χρήσιμος.

Οι λειτουργίες που είναι αναγκαίο να δηλωθούν στη γονική κλάση είναι η λήψη του κειμένου της ερώτησης και η αξιολόγηση της απάντησης. Οι λειτουργίες αυτές μπορούν να υλοποιηθούν στις παράγωγες, όχι όμως και στη γονική. Ως εκ τούτου, η γονική θα είναι αφηρημένη.

```
public abstract class Question {
    public abstract String getQuestion();
    public abstract boolean evaluateAnswer(String answer);
}
```

Κώδικας 13.21 *Η αφηρημένη κλάση Question.*

Στη συνέχεια, στον κώδικα 13.22, παρουσιάζεται η υλοποίηση της κλάσης των ερωτήσεων σωστού-λάθους.

```
public class TrueFalse extends Question {
    String text;
    boolean correctAnswer;

    public TrueFalse(String text, boolean answer) {
        this.text = text;
        this.correctAnswer = answer;
    }


    @Override
    public String getQuestion() {
        return text +"\n t(rue) or f(alse)\n";
    }
```

```
    @Override
    public boolean evaluateAnswer(String userAnswer) {
        return
String.valueOf(correctAnswer).toLowerCase().charAt(0)==userAnswer.toLowerCa
se().charAt(0);
    }
}
```

Κώδικας 13.22 *Η κλάση των ερωτήσεων true-false.*

Ο δημιουργός της κλάσης μας επιτρέπει να καθορίσουμε το κείμενο της ερώτησης και να προσδιορίσουμε ποια απάντηση είναι σωστή. Η getQuestion επιστρέφει μια συμβολοσειρά που αποτελείται από το κείμενο της ερώτησης ακολουθούμενης από τις επιλογές true ή false στην επόμενη από το κείμενο γραμμή. Η evaluateAnswer ελέγχει ώστε ο χαρακτήρας που έδωσε ως απάντηση ο χρήστης να είναι ίδιος με τον χαρακτήρα από τον οποίο αρχίζει η ορθή απάντηση της ερώτησης. Και οι δύο χαρακτήρες μετατρέπονται σε πεζούς ώστε η σύγκριση να γίνει αγνοώντας τυχόν διαφορές μεταξύ πεζών και κεφαλαίων.
Ο κώδικας 13.23 παρουσιάζει την κλάση ερωτήσεων πολλαπλής επιλογής.

```
import java.util.ArrayList;
import java.util.Arrays;

public class MultipleChoice extends Question {

    private final String text;
    private final ArrayList<String> answers;
    private final String correctAnswer;

    public MultipleChoice(String text,String correctAnswer,
String...answers) {
        this.text = text;
        this.answers=new ArrayList<>(Arrays.asList(answers));
        this.correctAnswer=correctAnswer;
    }

    @Override
    public String getQuestion() {
        StringBuilder rVal=new StringBuilder();
        rVal.append(text+"\n");
        char label='A';
        for (String answer:answers) {
            rVal.append(label).append(". ").append(answer).append("\n");
            label++;
        }
        return rVal.toString();
    }

    @Override
    public boolean evaluateAnswer(String userAnswer) {
        return correctAnswer.equalsIgnoreCase(userAnswer);
    }

    public String getCorrectAnswer() {
        return correctAnswer;
    }
}
```

Κώδικας 13.23 *Η κλάση των ερωτήσεων πολλαπλής επιλογής.*

Ο δημιουργός της κλάσης λαμβάνει ως παράμετρο το κείμενο της ερώτησης, την αρίθμηση της ορθής επιλογής και τις εναλλακτικές επιλογές. Προκειμένου να υπάρχει η δυνατότητα δημιουργίας ερωτήσεων με διαφορετικό πλήθος δυνητικών απαντήσεων, ο δημιουργός διαθέτει λίστα παραμέτρων μεταβλητού μήκους. Η getQuestion

αριθμεί τις εναλλακτικές επιλογές αυτόματα με κεφαλαίους λατινικούς χαρακτήρες αρχίζοντας από τον χαρακτήρα 'A' προς τον 'Z'. Επομένως, αν ο χρήστης της κλάσης δημιουργήσει ένα αντικείμενο MultipleChoice και τοποθετήσει πρώτη κατά σειρά τη σωστή απάντηση, στην παράμετρο του δημιουργού correctAnswer θα πρέπει να δώσει τον χαρακτήρα 'A', αν τοποθετήσει δεύτερη τη σωστή απάντηση θα πρέπει να δώσει τον χαρακτήρα 'B', κοκ. Ο αναγνώστης της σωστής απάντησης είναι απαραίτητος ώστε τα τεστ που χρησιμοποιούν τέτοιες ερωτήσεις να είναι σε θέση να πληροφορήσουν σχετικά τον χρήστη, όπως φαίνεται στον κώδικα 13.24 που ακολουθεί.

Έχοντας τις κλάσεις Question, TrueFalse και MultipleChoice είναι εύκολο να κατασκευάσουμε τεστ που βασίζονται σε αυτούς τους τύπους ερωτήσεων. Ο κώδικας 13.24 παρουσιάζει μία τέτοια κλάση που είναι σύμφωνη με τις απαιτήσεις της εκφώνησης.

```
import java.util.ArrayList;
import java.util.Scanner;

public class Quiz {

    static ArrayList<Question> test = new ArrayList<>();

    private static void setUpQuestions() {
        test.add(new TrueFalse("Η γάτα είναι κατοικίδιο", true));
        test.add(new TrueFalse("Ο σκύλος δεν εξημερώνεται", false));
        test.add(new MultipleChoice("Ο καλύτερος φίλος του ανθρώπου είναι
ο", "C", "όφις", "άλογο", "σκύλος"));
        test.add(new MultipleChoice("Η for είναι", "B", "δομή ελέγχου",
"επαναληπτική διαδικασία", "αναδρομική συνάρτηση"));
    }

    public static void main(String[] args) {
        Scanner in = new Scanner(System.in);
        setUpQuestions();
        for (Question q : test) {
            System.out.print(q.getQuestion());
            System.out.print("Εισάγετε την απάντηση: ");
            String userAnswer = in.next();
            if (q.evaluateAnswer(userAnswer)) {
                System.out.println("Σωστή απάντηση");
            } else if (q instanceof MultipleChoice) {
                System.out.println("Λάθος Απάντηση. Σωστή απάντηση η " +
((MultipleChoice) q).getCorrectAnswer());
            } else {
                System.out.println("Λάθος Απάντηση");
            }
            System.out.println();
        }

    }
}
```

Κώδικας 13.24 *Η κλάση των Quiz.*

Στην main καλείται καταρχάς η setUpQuestions που δημιουργεί και φορτώνει στο ArrayList test τις ερωτήσεις πολλαπλής επιλογής ή σωστού-λάθους. Στη συνέχεια κάθε μια ερώτηση τίθεται στον χρήστη με τη βοήθεια της getQuestion, λαμβάνεται η απάντηση του χρήστη η οποία αξιολογείται από την evaluateAnswer.

13.4.3 Μισθοδοσία Μηνός

Η εταιρεία CompSystems απασχολεί τριών τύπων εργαζόμενους, τους υπάλληλους γραφείου, τα διευθυντικά στελέχη και τους συμβασιούχους. Όλοι οι εργαζόμενοι στην εταιρεία είναι φυσικά πρόσωπα. Ως τέτοια κληρονομούν όλα τα χαρακτηριστικά της κλάσης Person (κώδικας 12.19). Οι αποδοχές των υπαλλήλων

γραφείου βασίζονται σε σταθερό μισθό 1200 Ευρώ τον μήνα. Επιπλέον, κάθε υπάλληλος γραφείου λαμβάνει μηνιαία ένα ποσό που υπολογίζεται ως το γινόμενο 5% x Αριθμός ετών στην εταιρεία x σταθερός μισθός.

Τα διευθυντικά στελέχη αμείβονται με σταθερό μισθό 3500 Ευρώ συν ένα ποσό που λαμβάνουν επίσης μηνιαία και έχει συμφωνηθεί μεταξύ εταιρείας και κάθε στελέχους ξεχωριστά.

Οι συμβασιούχοι αμείβονται με ένα ποσό ανά ώρα εργασίας που συμφωνείται μεταξύ της εταιρείας και κάθε συμβασιούχου ξεχωριστά. Κατά την έκδοση της μισθοδοσίας μηνός, οι συμβασιούχοι ενημερώνουν για τον αριθμό ωρών που εργάστηκαν κατά τη διάρκεια του μήνα.

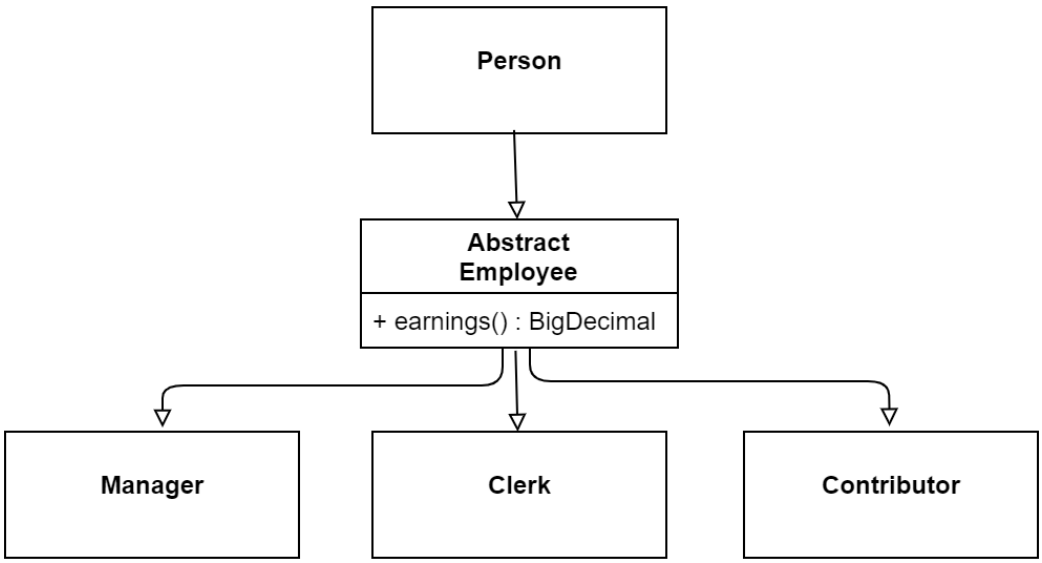
Η εταιρεία διαθέτει τρεις υπαλλήλους γραφείου, δύο διευθυντές και δύο συμβασιούχους. Οι διευθυντές έχουν συμφωνήσει ως επιπρόσθετο μηνιαίο ποσό 2000 Ευρώ ο ένας και 2200 ο άλλος. Οι συμβασιούχοι έχουν συμφωνήσει ωρομίσθιο 12 Ευρώ ο πρώτος και 20 ο δεύτερος. Ο πρώτος εργάστηκε 120 ώρες και ο δεύτερος 140 ώρες αυτόν τον μήνα. Τέλος, οι υπάλληλοι έχουν προσληφθεί σε τυχαίες ημερομηνίες μέσα στις 10.000 προηγούμενες ημερομηνίες από την τρέχουσα.

Η εταιρεία θέλει να υλοποιήσετε εφαρμογή που υπολογίζει και εμφανίζει κατάσταση μισθοδοσίας του τρέχοντος μήνα. Η κατάσταση σε κάθε γραμμή θα πρέπει να περιλαμβάνει τον αριθμό ταυτότητας του υπαλλήλου, το ονοματεπώνυμό του και τις αποδοχές του μήνα.

Λύση

Όπως διατυπώνεται στην εκφώνηση, οι εργαζόμενοι στην εταιρεία είναι φυσικά πρόσωπα. Ωστόσο, ως εργαζόμενοι στην εταιρεία πρέπει να είναι σε θέση να υπολογίζουν το μηνιαίο εισόδημά τους. Επομένως, επιτελούν μια επιπλέον λειτουργία, δηλαδή μια λειτουργία που δεν συναντάται σε πρόσωπα που δεν είναι εργαζόμενοι. Από την άλλη πλευρά, ο υπολογισμός του μηνιαίου εισοδήματος βασίζεται σε εντελώς διαφορετικά δεδομένα και είναι γενικότερα διαφορετικός για κάθε τύπο εργαζόμενου.

Με αυτά τα δεδομένα, μας εξυπηρετεί η οργάνωση των κλάσεων, όπως φαίνεται στο σχήμα 13.3.



Σχήμα 13.3 *Διάγραμμα κλάσεων της εφαρμογής Μισθοδοσία Μηνός*

Ο εργαζόμενος αναπαρίσταται από την κλάση Employee, παράγωγη κλάση της Person. Ωστόσο, πρόκειται για αφηρημένη κλάση, καθώς δηλώνει τη συνάρτηση earnings που επιστρέφει το μηνιαίο εισόδημα του εργαζόμενου. Η συνάρτηση αυτή όμως δεν μπορεί να οριστεί σαφώς στο επίπεδο του εργαζόμενου. Μπορεί όμως να οριστεί στις παράγωγες κλάσεις της Employee που αναπαριστούν εργαζόμενους συγκεκριμένου τύπου. Η κλάση Manager αναπαριστά τους διευθυντές, η κλάση Clerk τους υπαλλήλους γραφείου, η κλάση Contributor τους συμβασιούχους.

Ας δούμε καταρχάς την ανάπτυξη των κλάσεων και στη συνέχεια την εφαρμογή μισθοδοσία μηνός.

```
import java.math.BigDecimal;
import java.time.LocalDate;

public abstract class Employee extends Person {

    public Employee(String id, String fName, String sName, LocalDate birthday) {
        super(id, fName, sName, birthday);
    }

    public abstract BigDecimal earnings();
}
```

Κώδικας 13.25 *Η αφηρημένη κλάση Employee*

Η κλάση Employee έχει ένα ιδιαίτερο χαρακτηριστικό. Κληρονομεί την κλάση Person και ταυτόχρονα είναι αφηρημένη. Παρότι δεν είναι εφικτή η δημιουργία αντικειμένων της κλάσης, απαιτείται να οριστεί δημιουργός ο οποίος εκείνο που κάνει είναι ότι καλεί τον δημιουργό της γονικής Person. Ο δημιουργός εδώ είναι απαραίτητος ώστε οι παράγωγες κλάσεις Manager, Clerk και Contributor να μπορούν να τον καλέσουν και αυτός με τη σειρά του να καλέσει τον δημιουργό Person. Κατά τα λοιπά, η κλάση δηλώνει την αφηρημένη συνάρτηση earnings που υπολογίζει το μηνιαίο εισόδημα του εργαζόμενου.

```
import java.math.BigDecimal;
import java.time.LocalDate;

public class Manager extends Employee {
    private final BigDecimal constantFee=new BigDecimal(3500);
    private final BigDecimal bonus;

    public Manager(BigDecimal bonus, String id, String fName, String sName,
LocalDate birthday) {
        super(id, fName, sName, birthday);
        this.bonus = bonus;
    }

    @Override
    public BigDecimal earnings() {
        return constantFee.add(bonus);
    }
}
```

Κώδικας 13.26 *Η κλάση Manager*

Ο δημιουργός της κλάσης Manager καλεί τον δημιουργό της Employee και στη συνέχεια δίνει τιμή στην πρόσθετη αμοιβή του στελέχους. Σύμφωνα με την εκφώνηση, η πρόσθετη αμοιβή καθορίζεται σε συμφωνία με την εταιρεία ξεχωριστά για κάθε στέλεχος. Επομένως, μπορεί η τιμή αυτή να καθοριστεί κατά τη δημιουργία των αντικειμένων Manager, ειδικά για κάθε αντικείμενο.

Στη συνέχεια, υλοποιείται η συνάρτηση earnings που απλώς επιστρέφει το άθροισμα του μηνιαίου μισθού συν την πρόσθετη αμοιβή.

Η κλάση Clerk παρουσιάζεται στον κώδικα 13.27:

```
import java.math.BigDecimal;
import java.time.LocalDate;
import java.time.Period;
import java.util.Random;

public class Clerk extends Employee {
    private static final BigDecimal BASESALARY=new BigDecimal(1200);
    private static final double INCREMENTRATE=0.05;
```

```
    private final LocalDate hireDate;

    public Clerk(String id, String fName, String sName, LocalDate birthday)
{
        super(id, fName, sName, birthday);
        hireDate=LocalDate.now().minusDays(new Random().nextInt(10000));
    }


    @Override
    public BigDecimal earnings() {
        LocalDate today = LocalDate.now();
        int workingYears=Period.between(hireDate, today).getYears();
        BigDecimal incrementAmount=BASESALARY.multiply(new
BigDecimal(workingYears*INCREMENTRATE));
        double d=incrementAmount.doubleValue();
        BigDecimal salary=BASESALARY.add(incrementAmount);
        return salary;
    }
}
```

Κώδικας 13.27 *Η κλάση Clerk*

Οι υπάλληλοι γραφείου αμείβονται με σταθερό μηνιαίο μισθό συν μια προσαύξηση που εξαρτάται από ένα σταθερό ποσοστό και από τα έτη προϋπηρεσίας στην εταιρεία. Εφόσον ο μηνιαίος μισθός και το ποσοστό προσαύξησης είναι σταθερά ποσά, οι μεταβλητές BASESALARY και INCREMENTRATE δηλώθηκαν ως final. Επιπλέον, τόσο ο μηνιαίος μισθός όσο και το ποσοστό προσαύξησης είναι κοινά για όλους τους υπαλλήλους γραφείου, επομένως, οι αντίστοιχες μεταβλητές θα πρέπει να δηλωθούν ως static, καθώς αφορούν την κλάση και όχι κάθε αντικείμενό της χωριστά. Η ημερομηνία πρόσληψης όμως αφορά κάθε εργαζόμενο χωριστά και δηλώνεται ως μεταβλητή στιγμιότυπου. Καθώς και αυτή δεν είναι λογικό να αλλάξει μετά την πρόσληψη εργαζόμενου, μπορεί επίσης να δηλωθεί ως final.

Ο δημιουργός της κλάσης, αφού καλέσει τον δημιουργό της γονικής, δίνει τιμή στην ημερομηνία πρόσληψης σύμφωνα με τις απαιτήσεις της εκφώνησης.

Στη συνέχεια, η κλάση επανορίζει τη συνάρτηση earnings. Υπολογίζει τα έτη από την ημερομηνία πρόσληψης μέχρι σήμερα, μετά, το ποσό προσαύξησης, το οποίο προσθέτει στον βασικό μισθό και επιστρέφει τις αποδοχές του μήνα.

Τέλος, οι συμβασιούχοι αναπαρίστανται από την κλάση Contributor (κώδικας 13.28):

```
import java.math.BigDecimal;
import java.time.LocalDate;

public class Contributor extends Employee {

    private int workingHours;
    private BigDecimal paymentPerHour;

    public Contributor(BigDecimal paymentPerHour, String id, String fName,
String sName, LocalDate birthday) {
        super(id, fName, sName, birthday);
        this.paymentPerHour = paymentPerHour;
    }

    public void setWorkingHours(int workingHours) {
        this.workingHours = workingHours;
    }

    @Override
    public BigDecimal earnings() {
        return paymentPerHour.multiply(new BigDecimal(workingHours));
    }
```

```
}
```

Κώδικας 13.28 *Η κλάση Contributor*

Οι συμβασιούχοι αμείβονται με βάση μια αμοιβή για την ώρα εργασίας τους που συμφωνείται μεταξύ εταιρείας και κάθε συμβασιούχου χωριστά. Η πληροφορία συνοδεύει κάθε συμβασιούχο από τη δημιουργία του και επομένως μπορεί να δοθεί στον δημιουργό. Οι ώρες εργασίας όμως αφορούν τις ώρες εργασίας του μήνα, πιθανότατα παρουσιάζουν διακυμάνσεις από μήνα σε μήνα και άρα δεν σχετίζονται με τη δημιουργία του αντικειμένου. Έτσι παρέχουμε τον ρυθμιστή setWorkingHours ώστε να ενημερώνεται κατάλληλα η αντίστοιχη μεταβλητή. Τέλος, η earnings επιστρέφει το γινόμενο ώρες εργασίας επί αμοιβή ανά ώρα.

Έχοντας διαθέσιμες τις κλάσεις, η ανάπτυξη της εφαρμογής είναι σχετικά εύκολη.

```
import java.math.BigDecimal;
import java.text.DecimalFormat;
import java.text.DecimalFormatSymbols;
import java.time.LocalDate;
import java.time.Month;
import java.util.ArrayList;

public class CalcMonthlyPayment { //Κώδικας 13.29

    public static void main(String[] args) {
        ArrayList<Employee> personel = new ArrayList<>();
        personel.add(new Clerk("1", "Jim", "Black", LocalDate.of(1967,
Month.MARCH, 1)));
        personel.add(new Clerk("2", "John", "Blue", LocalDate.of(1972,
Month.APRIL, 12)));
        personel.add(new Clerk("3", "Jack", "Dalton", LocalDate.of(1980,
Month.MARCH, 21)));
        personel.add(new Manager(BigDecimal.valueOf(2000), "101", "Adam",
"Smith", LocalDate.of(1980, Month.JANUARY, 12)));
        personel.add(new Manager(BigDecimal.valueOf(2200), "102", "Adam",
"Smith", LocalDate.of(1980, Month.MARCH, 21)));
        Contributor cntr = new Contributor(BigDecimal.valueOf(12), "1001",
"Kalamity", "Jane", LocalDate.of(2000, Month.MARCH, 21));
        cntr.setWorkingHours(120);
        personel.add(cntr);
        cntr = new Contributor(BigDecimal.valueOf(20), "1002", "Erick",
"Erickson", LocalDate.of(2001, Month.MARCH, 21));
        cntr.setWorkingHours(140);
        personel.add(cntr);

        DecimalFormatSymbols dfs = new DecimalFormatSymbols();
        dfs.setDecimalSeparator('.');
        dfs.setGroupingSeparator(',');
        DecimalFormat f = new DecimalFormat("######.###");
        f.setDecimalFormatSymbols(dfs);

        for (Employee e : personel) {
            System.out.println(e.getId() + " " + e.getfName() + " " +
e.getsName() + " " + f.format(e.earnings().doubleValue()));
        }
    }
}
```

Κώδικας 13.29 *Η εφαρμογή CalcMonthlyPayment*

Καταρχάς, η εφαρμογή δημιουργεί ένα ArrayList για Employee. Στη συνέχεια δημιουργεί ένα αντικείμενο για κάθε εργαζόμενο και το προσθέτει στο ArrayList. Ειδικά για τα αντικείμενα Contributor πριν τα προσθέσει στο ArrayList, ενημερώνει τις ώρες εργασίας. Στη συνέχεια, φροντίζει για την κατάλληλη διαμόρφωση της εξόδου και μετά τυπώνει την κατάσταση μισθοδοσίας βασιζόμενη απλώς στις συναρτήσεις των κλάσεων. Οι getId,

getfName και getsName έχουν κληρονομηθεί αυτούσιες από την κλάση Person. Η earnings, όπως είδαμε, ορίζεται στις σαφείς κλάσεις εργαζόμενων. Προκειμένου να αξιοποιήσουμε την format, μετατρέπουμε τα earnings από BigDecimal σε double.

13.5 Ασκήσεις προς Λύση

13.5.1 Ο ρόμβος

Στην άσκηση 13.4.1 υπολογισμού εμβαδού γεωμετρικών σχημάτων να υποστηριχτεί ο υπολογισμός εμβαδού του ρόμβου. Ο υπολογισμός του εμβαδού ρόμβου μπορεί να γίνει συνάρτηση των μηκών των διαγώνιων του σύμφωνα με τον τύπο που ακολουθεί

$$E = \frac{(\delta 1 \times \delta 2)}{2}, \delta 1 \; και \; \delta 2 \; οι \; διαγώνιοι$$

Επιπλέον, προσθέστε στις κλάσεις της άσκησης όλες τις αναγκαίες μεθόδους που έχουν παραληφθεί, π.χ. equals, hashCode, toString, ρυθμιστές και αναγνώστες.

13.5.2 Ζωολογικός κήπος

Σε έναν ζωολογικό κήπο έχουν συγκεντρωθεί διάφορα ζώα. Από αυτά, κάποια περπατούν, κάποια πετούν, κάποια κολυμπούν και κάποια έρπουν. Κατασκευάστε έναν ζωολογικό κήπο, προσθέστε ζώα από κάθε τύπο και θέστε τα σε κίνηση. Οι συναρτήσεις κίνησης θα βγάζουν μηνύματα που απλώς περιγράφουν την κίνηση, π.χ. το λιοντάρι περπατά, το δελφίνι κολυμπά.

13.5.3 Η συμπεριφορά των μεταβλητών αναφοράς

Στην άσκηση 13.4.3 ενημερώνουμε πρώτα τις ώρες εργασίας ενός αντικειμένου τύπου Contributor και στη συνέχεια προσθέτουμε το αντικείμενο στο ArrayList. Τι θα συμβεί αν κάνουμε το αντίθετο, δηλαδή αν προσθέσουμε στο ArrayList το αντικείμενο και στη συνέχεια ενημερώσουμε τις ώρες εργασίας; Ελέγξτε και εξηγήστε τη συμπεριφορά.

13.5.4 Βαθμολογία

Σε ένα τμήμα της Γ' Γυμνασίου φοιτούν 11 μαθητές. Από αυτούς, οι τρεις είναι αλλοδαποί και οι υπόλοιποι ημεδαποί. Όλοι οι μαθητές παρακολουθούν τα μαθήματα Άλγεβρας, Φυσικής και Γλώσσας. Οι αλλοδαποί παρακολουθούν επιπλέον το μάθημα Εκμάθηση της Ελληνικής γλώσσας. Όλοι οι μαθητές μπορούν να παρακολουθήσουν ή όχι Θρησκευτικά. Το μάθημα έχει επιλέξει ένας αλλοδαπός και έξι ημεδαποί. Όλοι οι μαθητές είναι φυσικά πρόσωπα. Βρισκόμαστε στο τέλος της περιόδου των μαθημάτων, οι μαθητές έχουν βαθμολογηθεί και πρέπει να υπολογίσουμε τον μέσο όρο βαθμολογίας κάθε μαθητή. Φορτώστε σε ένα ArrayList τους 11 μαθητές και εμφανίστε για κάθε έναν τον αριθμό ταυτότητας, το ονοματεπώνυμό του, την αναλυτική βαθμολογία του και τον μέσο όρο του.

Βιβλιογραφία

[1] M. Fowler, UML Distilled: A Brief Guide to the Standard Object Modeling Language, 3rd edition. Boston: Addison-Wesley Professional, 2003.

Κεφάλαιο 14

Σύνοψη
Σε αυτήν την ενότητα καταρχάς εξηγείται η έννοια της διεπαφής στην Java. Στη συνέχεια παρουσιάζονται βασικές προκαθορισμένες διεπαφές και η χρήση τους. Αναλύεται η υλοποίηση διεπαφών οριζόμενων από τον χρήστη. Συζητούνται τα προβλήματα της πολλαπλής κληρονομικότητας και αναλύεται ο μηχανισμός μέσα από τον οποίον υποστηρίζει η Java την πολλαπλή κληρονομικότητα. Εξηγείται η κληρονομικότητα ανάμεσα σε διεπαφές, η ταυτόχρονη κληρονομικότητα από κλάση και διεπαφή, τα στατικά μέλη στις διεπαφές, οι προκαθορισμένες μέθοδοι και οι ανώνυμες κλάσεις.

Προαπαιτούμενη γνώση
Η ενσωμάτωση, η κληρονομικότητα και ο πολυμορφισμός στο αντικειμενοστρεφές μοντέλο με Java, οι αφηρημένες κλάσεις καθώς και οι βασικές προκαθορισμένες κλάσεις, δηλαδή οι String, StringBuilder, ArrayList, LinkedList, Stack, PriorityQueue, HashSet, HashMap, HashTree, LocalDate, Localtime, Period και Duration.

Λέξεις κλειδιά
Διεπαφή (interface), πρόβλημα του διαμαντιού (the diamond problem), πολλαπλή κληρονομικότητα (multiple inheritance), κληρονομικότητα κατάστασης (inheritance of state), κληρονομικότητα υλοποίησης (inheritance of inheritance), κληρονομικότητα τύπου (state inheritance), προκαθορισμένες μέθοδοι (default methods), ανώνυμες κλάσεις (anonymous class).

14 Διεπαφές

Σε πολλές περιπτώσεις, μια ομάδα μηχανικών λογισμικού χρειάζεται να χρησιμοποιήσει λογισμικό που έχει αναπτύξει άλλη ομάδα. Στις περιπτώσεις αυτές χρειάζεται ένας τυπικός τρόπος με τον οποίο η δεύτερη ομάδα πληροφορεί την πρώτη για τα χαρακτηριστικά του λογισμικού της. Αυτός ο τρόπος παρέχεται δια μέσου των διεπαφών (interfaces) [1]. Με άλλα λόγια, όταν ένα λογισμικό Β καλεί ένα λογισμικό Α, τότε το Β χρειάζεται να γνωρίσει τα χαρακτηριστικά του Α. Αυτά ακριβώς τα χαρακτηριστικά εκθέτει το Α προς τους τρίτους δια μέσου των διεπαφών. Πρόκειται δηλαδή για μια συμφωνία μεταξύ των δύο ομάδων μηχανικών λογισμικού ή ακριβέστερα μεταξύ δύο λογισμικών με την οποία καθορίζονται τα χαρακτηριστικά του λογισμικού ώστε η ομάδα-χρήστης να μπορεί να το χρησιμοποιεί χωρίς να γνωρίζει τις εσωτερικές λεπτομέρειές του. Με αυτήν την έννοια, οι διεπαφές αποτελούν έναν ακόμη μηχανισμό αφαιρετικότητας. Αρχικά, οι διεπαφές της Java συνιστούσαν ένα είδος απολύτως αφηρημένης κλάσης, δηλαδή μίας κλάσης στην οποία όλες οι μέθοδοι στιγμιότυπου ήταν αφηρημένες. Από την έκδοση 8 και μετά προστέθηκαν οι προκαθορισμένες μέθοδοι (default methods) που είναι σαφείς. Οι διεπαφές εκτός από αφηρημένες και προκαθορισμένες μεθόδους μπορεί να περιέχουν και στατικές μεθόδους, σταθερές και εμφωλευμένους τύπους. Δεν μπορούν όμως να περιέχουν μεταβλητές στιγμιότυπου. Οι διεπαφές είναι τύποι αναφοράς, ωστόσο δεν μπορεί να δημιουργηθούν στιγμιότυπα διεπαφής για τον ίδιο λόγο που δεν μπορεί να δημιουργηθούν στιγμιότυπα αφηρημένης κλάσης. Μπορούν όμως να δημιουργηθούν στιγμιότυπα των κλάσεων που υλοποιούν (implements) τις διεπαφές. Κάθε κλάση που υλοποιεί μια διεπαφή συνδέεται με τη διεπαφή με τη σχέση “είναι ένα”.

Οι διεπαφές μπορούν να κληρονομούν άλλες διεπαφές. Μάλιστα μια διεπαφή μπορεί να έχει πολλαπλές γονικές διεπαφές. Οι διεπαφές όμως δεν μπορούν να κληρονομήσουν μία κλάση.

Ένα άλλο χαρακτηριστικό των διεπαφών είναι πως για κάθε δημόσια μέθοδο της κλάσης Object, μία αφηρημένη μέθοδος με ίδια ταυτότητα δηλώνεται αυτόματα σε κάθε διεπαφή. Για παράδειγμα, αν δούμε την τεκμηρίωση μιας προκαθορισμένης διεπαφής List θα βρούμε τη μέθοδο boolean equals(Object o). Δημιουργείται έτσι λανθασμένα η εντύπωση πως οι διεπαφές κληρονομούν την κλάση Object. Γενικά οι διεπαφές δεν κληρονομούν από κλάσεις. Ωστόσο, μια διεπαφή μπορεί να κληρονομεί από μία ή περισσότερες διεπαφές.

Πέραν των ανωτέρω, οι διεπαφές αποτελούν τον μηχανισμό μέσα από τον οποίον η Java επιτυγχάνει την πολλαπλή κληρονομικότητα τύπου ενώ ενισχύουν τις δυνατότητες πολυμορφικού κώδικα.

Όλα αυτά θα τα δούμε στη συνέχεια αναλυτικά, αρχίζοντας με την παρουσίαση κάποιων βασικών διεπαφών της βιβλιοθήκης της Java.

14.1 Βασικές διεπαφές

Στις βιβλιοθήκες της Java πέραν των κλάσεων έχουν ορισθεί και πάρα πολλές διεπαφές. Παρουσιάζουμε εδώ συγκεκριμένα τη διεπαφή Iterator και Comparator. Αρχικά μας ενδιαφέρουν κυρίως οι αφηρημένες μέθοδοί τους. Παρακάτω σε αυτό το κεφάλαιο θα συζητήσουμε για τα υπόλοιπα χαρακτηριστικά των διεπαφών.

14.1.1 Iterator

Ο Iterator είναι ίσως η συχνότερα χρησιμοποιούμενη διεπαφή. Μας δίνει τη δυνατότητα να προσπελάσουμε ένα προς ένα τα στοιχεία μιας συλλογής. Δεδομένου ότι οι συλλογές αποτελούν γενικεύσεις, έτσι και ο Iterator αποτελεί μια γενίκευση, π.χ. για να προσπελάσουμε μέσω ενός Iterator τα στοιχεία ενός ArrayList<Person>, χρειαζόμαστε έναν Iterator<Person>. Γενικά, κάθε Iterator αφορά κάποια συλλογή.
Ο Iterator ορίζει δύο αφηρημένες μεθόδους:

```
boolean hasNext()
```

Ελέγχει αν υπάρχει επόμενο στοιχείο στη συλλογή

```
E next()
```

Επιστρέφει το επόμενο διαθέσιμο στοιχείο της συλλογής.

Κάθε μια από τις προκαθορισμένες συλλογές διαθέτει μια μέθοδο που επιστρέφει έναν Iterator με τον οποίο μπορούμε να προσπελάσουμε τα στοιχεία της. Ακολουθεί ένα παράδειγμα χρήσης Iterator με ArrayList.

```
import gr.ihu.cs.lmous.OOJ.k12.v1.x.Person;
import java.time.LocalDate;
import java.time.Month;
import java.util.ArrayList;
import java.util.Iterator;

public class IteratorDemo {

    public static ArrayList<Person> loadData() {
        ArrayList<Person> rVal = new ArrayList<>();
        rVal.add(new Person("10", "Jim", "Smith", LocalDate.of(1961,
Month.JANUARY, 1)));
        rVal.add(new Person("20", "Adam", "Smith", LocalDate.of(1965,
Month.JANUARY, 1)));
        rVal.add(new Person("1", "John", "Black", LocalDate.of(1971,
Month.JANUARY, 1)));
        rVal.add(new Person("2", "John", "Dalton", LocalDate.of(1965,
Month.JANUARY, 1)));
        rVal.add(new Person("12", "Bill", "Adams", LocalDate.of(1965,
Month.JANUARY, 5)));
        rVal.add(new Person("6", "Averell", "Dalton", LocalDate.of(1972,
Month.JANUARY, 1)));
        rVal.add(new Person("7", "John", "Cyan", LocalDate.of(1980,
Month.JANUARY, 1)));
        return rVal;
    }

    public static void main(String[] args) {
        ArrayList<Person> store = loadData();
        Iterator<Person> it = store.iterator();
        while (it.hasNext()) {
            System.out.println(it.next());
        }
    }
}
```

Κώδικας 14.1 *Παράδειγμα χρήσης Iterator με ArrayList.*

Στην main του κώδικα 14.1 καλείται καταρχάς η loadData που φορτώνει στο ArrayList, store, τα δεδομένα 7 προσώπων. Η κλάση ArrayList διαθέτει τη μέθοδο iterator που επιστρέφει έναν Iterator για πρόσωπα. Καλώντας την iterator λαμβάνουμε έναν τέτοιον Iterator και τον τοποθετούμε στην αναφορά it. Διαμέσου αυτής της αναφοράς προσπελαύνουμε ένα-ένα τα στοιχεία της ArrayList με την it.next(). Αυτή θα πρέπει να καλείται μόνον εφόσον έχουμε ελέγξει πως υπάρχει διαθέσιμο επόμενο στοιχείο, δηλαδή μόνον εφόσον η it.hasNext() επιστρέψει true. Να σημειωθεί πως σε αυτό το παράδειγμα, η it.next() επιστρέφει ένα αντικείμενο τύπου Person.

Ας δούμε και ένα παράδειγμα που θα βοηθήσει να κατανοήσουμε τις δυνατότητες πολυμορφικού κώδικα που παρέχονται μέσω των διεπαφών.

```
import java.time.LocalDate;
import java.time.Month;
import java.util.ArrayList;
import java.util.Iterator;
import java.util.LinkedList;
import java.util.List;

public class PolymorphismDemo {

      public static List<Person> loadData() {
        List<Person> rVal = new ArrayList<>();
        rVal.add(new Person("10", "Jim", "Smith", LocalDate.of(1961,
Month.JANUARY, 1)));
        rVal.add(new Person("20", "Adam", "Smith", LocalDate.of(1965,
Month.JANUARY, 1)));
        rVal.add(new Person("1", "John", "Black", LocalDate.of(1971,
Month.JANUARY, 1)));
        rVal.add(new Person("2", "John", "Dalton", LocalDate.of(1965,
Month.JANUARY, 1)));
        rVal.add(new Person("12", "Bill", "Adams", LocalDate.of(1965,
Month.JANUARY, 5)));
        rVal.add(new Person("6", "Averell", "Dalton", LocalDate.of(1972,
Month.JANUARY, 1)));
        rVal.add(new Person("7", "John", "Cyan", LocalDate.of(1980,
Month.JANUARY, 1)));
        return rVal;
    }

    public static void main(String[] args) {
        List<Person> store = loadData();
        for (int i=0; i<2; i++) {
            Iterator<Person> it = store.iterator();
            while (it.hasNext()) {
                System.out.println(it.next());
            }
            System.out.println("-------------------------------------");
            store = new LinkedList<>(store);
        }
    }
}
```

Κώδικας 14.2 *Προσπέλαση των στοιχείων μιας ArrayList και μιας LinkedList με τον ίδιο κώδικα*

Στον κώδικα 14.2 η loadData έχει τροποποιηθεί ώστε ο τύπος επιστροφής της δεν είναι ArrayList<Person> όπως στον κώδικα 14.1 αλλά List<Person>. Η List είναι προκαθορισμένη διεπαφή. Τόσο η κλάση ArrayList όσο και η κλάση LinkedList υλοποιούν τη διεπαφή List. Αυτό σημαίνει πως μια ArrayList είναι μια List και μια LinkedList είναι επίσης μία List. Επομένως, η loadData που η τυπική τιμή επιστροφής της είναι List<Person> μπορεί να επιστρέψει πραγματικό αντικείμενο τύπου ArrayList<Person> ή LinkedList<Person> ή οποιασδήποτε άλλης συλλογής που υλοποιεί την List.